

REST API, JSON, Auto-Docs & Testing

Web Design and Programming

M.Sc. Le Duy Khanh

Faculty of Data Science and Artificial Intelligence

Session Outline

1. RESTful API Design
2. JSON Serialization
3. OpenAPI & Auto-Docs
4. CORS for the Frontend
5. Testing with pytest
6. Frontend Calls the Heroes API
7. API Quality
8. Strategy for DS & AI Students
9. Worked Example & Labs
10. Review & Summary

Learning Objectives

By the end of this session, you will be able to:

1. Apply **RESTful design**: plural-noun resources, HTTP verbs, stateless, correct **status codes**.
2. Distinguish **REST** from **RPC** URLs (`GET /heroes/5` vs `GET /getHero?id=5`).
3. Explain how **Pydantic** validates and serializes JSON; control output with `response_model`.
4. Use **OpenAPI** (`/openapi.json`), **Swagger UI** (`/docs`), **ReDoc** (`/redoc`); improve them with `tags`, `summary`, examples.
5. Configure `CORSMiddleware` for a frontend on another origin.
6. Test the API with `pytest` + `TestClient`: fixtures, test DB, dependency overrides, 400/404/409/422 cases.
7. Call the API from the frontend with `fetch()`; apply pagination and versioning (`/api/v1`).

Where We Are: The Heroes API

- Week 5: $\mathrm{fetch}()$ in the browser – and the $\mathrm{blocked\ by\ CORS\ policy}$ error.
- Week 9: the **Heroes API** – $\mathrm{Team} + \mathrm{Hero}$ tables, SQLModel, CRUD endpoints.
- This week: **same project**, turned into a proper REST API:
  - correct URLs, verbs, status codes; validated JSON
  - self-generated docs at $/\mathrm{docs}$
  - CORS fixed on the server
  - automated tests with $\mathrm{pytest} + \mathrm{TestClient}$
  - a web page that lists and creates heroes
- Result: **Frontend** $\leftrightarrow$ **Backend loop closed**.

RESTful API Design

What Is REST?

- An **API is a contract** between frontend and backend.
- **REST** (Representational State Transfer, Roy Fielding) – an *architectural style*:
  - HTTP, **client-server**
  - **Stateless** – each request carries everything; server remembers nothing between requests
  - **Cacheable**
  - **Resource-based** – URL names the *thing*, HTTP verb names the *action*
- Heroes API resources: **heroes** and **teams** (the Week 9 tables).
- **Endpoint** (route) = URL + HTTP verb → one function.

Resources & Verbs: Heroes

Action	Verb	URL	Success
List heroes	GET	/heroes	200 OK
Read one hero	GET	/heroes/{id}	200 OK
Create a hero	POST	/heroes	201 Created
Replace a hero	PUT	/heroes/{id}	200 OK
Partially update	PATCH	/heroes/{id}	200 OK
Delete a hero	DELETE	/heroes/{id}	204 No Content

- Plural nouns, **consistent** everywhere. We use PATCH only (no PUT).
- Filter / paginate with **query parameters**: /heroes?team_id=1&offset=20&limit=10
- **Nested** child resource: /teams/1/heroes – unknown team ⇒ 404 (a filter just gives []).

REST vs RPC: The Heroes API Done Wrong

RPC-style (avoid)	RESTful
GET /getHero?id=5	GET /heroes/5
GET /getAllHeroes	GET /heroes
POST /createHero	POST /heroes
POST /updateHeroAge	PATCH /heroes/5 {”age”: 46}
GET /heroes/5/delete	DELETE /heroes/5
POST /assignHeroToTeam?hero=5&team=2	PATCH /heroes/5 {”team_id”: 2}
POST /retireHero/5	PATCH /heroes/5 {”status”: ”retired”}

Rule of thumb

Remove the verb from the URL – if the HTTP method still says what happens, it is RESTful. “Actions” like assign/retire are usually a PATCH of one field.

Status Codes in the Heroes API

Code	Name	When
200	OK	GET /heroes, PATCH /heroes/5
201	Created	POST /heroes
204	No Content	DELETE /heroes/5 – no body
400	Bad Request	POST /heroes with team_id that does not exist
401	Unauthorized	Not **authenticated** (Week 11)
403	Forbidden	Authenticated but **not allowed** (Week 11)
404	Not Found	GET /heroes/999
409	Conflict	POST /teams with an existing name
422	Unprocessable Entity	Body fails **Pydantic validation**, e.g. "age": -5
500	Internal Server Error	Bug / failure on the server

2xx success 4xx **client** error 5xx **server** error

Setting Status Codes in FastAPI

```python
from fastapi import HTTPException, status

@router.post("", status_code=status.HTTP_201_CREATED)
def create_hero(hero_in: HeroCreate, session: SessionDep): ...

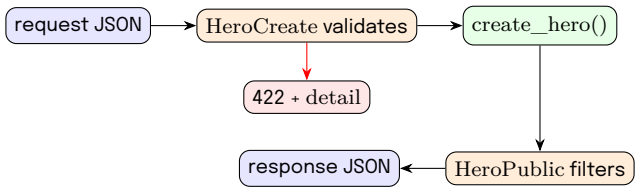
@router.delete("/{hero_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_hero(hero_id: int, session: SessionDep): ...

# error path: overrides the success code
raise HTTPException(status_code=404, detail="Hero not found")
```

- Default success code is 200; status.* constants are readable and typo-proof.
- Same error shape everywhere: {"detail": "..."}.
- Same hero shape everywhere: every endpoint uses response_model=HeroPublic.

JSON Serialization

Pydantic Does Both Directions



- **Input**: types & constraints checked before your code runs – failure ⇒ automatic **422**.
- **Output**: the `Hero` you return is filtered by `response_model`, then encoded to JSON.

Input Validation → 422

```python
class HeroBase(SQLModel):      # Week 9 had types only; now: rules
    name: str = Field(index=True, min_length=1, max_length=50)
    age: int | None = Field(default=None, ge=0, le=1000)
    team_id: int | None = Field(default=None, foreign_key="team.id")
```

POST /api/v1/heroes {..., "age": -5} returns:

```
{"detail": [{"type": "greater_than_equal",
             "loc": ["body", "age"],
             "msg": "Input should be greater than or equal to 0",
             "input": -5, "ctx": {"ge": 0}}]}
```

- loc tells the frontend **which field** is wrong.
- Validation runs on HeroCreate/HeroUpdate (request bodies), not the table=True model.

Output: response_model

- Fields not in the response model are **dropped** – HeroPublic hides secret_name.

Decorator option	**Effect**
response_model_exclude_none=True	Omit None fields (hero without age)
response_model_exclude_unset=True	Omit fields never set
response_model_exclude={"team_id"}	Drop listed fields
response_model_include={"id","name"}	Keep only listed fields

- Prefer a separate **output model** – it documents the contract in /docs.

New Hero Fields: status (Enum) & created_at

```python
def utc_now() -> datetime:
    return datetime.now(timezone.utc)          # timezone-aware

class HeroStatus(str, Enum):
    active = "active"
    retired = "retired"

class HeroBase(SQLModel):
    ...
    status: HeroStatus = HeroStatus.active      # client may send it
class Hero(HeroBase, table=True):
    ...
    created_at: datetime = Field(default_factory=utc_now)  # server sets it
class HeroPublic(HeroBase):
    id: int
    created_at: datetime
```

```
{"name": "Tony", "age": 45, "status": "active", "team_id": 1,
 "id": 1, "created_at": "2026-10-05T09:30:00.123456Z"}
```

- Enum → its value; {"status": "dead"} ⇒ 422. datetime → ISO 8601.
- **Schema change**: create_all won't alter hero – delete app.db (dev) or Alembic migration (Week 9).

OpenAPI & Auto-Docs

Docs Generated From Code

URL	What	Use for
/openapi.json	Raw OpenAPI spec	Tools: codegen, Postman, test generators
/docs	**Swagger UI**	Interactive: Try it out → Execute
/redoc	**ReDoc**	Read-only reference

type hints + models → OpenAPI → /docs, /redoc (and generated tests)

- Docs **never go out of date**: add status → save → refresh /docs → schema + drop-down active/retired appear.

Making /docs Useful: App, Endpoint, Field

```python
app = FastAPI(title="Heroes API", version="1.0.0", openapi_tags=[
    {"name": "heroes", "description": "Create, read, update and delete heroes."},
    {"name": "teams", "description": "Teams and the heroes that belong to them."}])

router = APIRouter(prefix="/heroes", tags=["heroes"])      # one section per resource

@router.get("/{hero_id}", response_model=HeroPublic,
            summary="Get one hero",                                  # one-line title
            responses={404: {"description": "Hero not found"}})
def read_hero(hero_id: int, session: SessionDep):
    """Return a single hero by its **id**."""                        # long description
    return get_hero_or_404(session, hero_id)

class HeroBase(SQLModel):                                            # pre-filled "Try it out"
    name: str = Field(index=True, min_length=1, max_length=50,
                      schema_extra={"examples": ["Deadpond"]})
# plain Pydantic model: Field(examples=["Deadpond"])
```

CORS for the Frontend

Origin = Scheme + Host + Port

Page	API	Same origin?
http://127.0.0.1:8000 (/docs)	http://127.0.0.1:8000/api/v1/heroes	yes
http://127.0.0.1:5500 (Live Server)	http://127.0.0.1:8000/api/v1/heroes	**no** – port
http://localhost:5500	http://127.0.0.1:8000/api/v1/heroes	**no** – host, port
https://heroes-ui.example.com	https://heroes-api.example.com/…	**no** – host

- /docs always works (same origin); our hero page on :5500 is **cross-origin**.
- Browser **same-origin policy**: JS cannot read another origin's response unless the **server allows it**.
- JSON POST /heroes ⇒ browser sends a **preflight** OPTIONS first.
- Only **browsers** enforce CORS – curl, Postman, TestClient are never blocked.

CORSMiddleware: **Fix It on the Server**

```python
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5500",
                   "http://127.0.0.1:5500"],   # + deployed UI (Week 13)
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- Exact origins, **no trailing slash**.
- **Never** allow_origins=["*"] with allow_credentials=True: not rejected – any site could send your users' cookies.
- Read origins from an env var in deployment (like DATABASE_URL).
- This fixes the Week 5 blocked by CORS policy error.

Testing with pytest

pytest Essentials

- Clicking through /docs after every change doesn't scale – did you re-check all 5 hero endpoints after adding status?
- pip install pytest httpx (TestClient is built on httpx).
- **Discovery**: test_*.py files, test_* functions run automatically.
- Plain assert – failures show expected vs actual.
- **Fixtures**: reusable setup injected by **parameter name** (like Depends).
- **Parametrize**: one test, many inputs.
- TestClient(app): calls the app **in-process** – no server, no port.

First Test – and a Problem

```python
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create_hero():
    r = client.post("/api/v1/heroes",
                    json={"name": "Deadpond", "secret_name": "Dive Wilson"})
    assert r.status_code == 201
    assert r.json()["name"] == "Deadpond"
```

- client.get/post/patch/delete; json= for body, params= for query.
- **Problem**: get_session uses the **real** DB – every run adds another Deadpond to app.db.
- Tests must **never touch real data**.

Test DB + Dependency Override (conftest.py)

```python
from sqlmodel import SQLModel, Session, create_engine
from sqlmodel.pool import StaticPool
from app.main import app
from app.database import get_session

@pytest.fixture(name="session")
def session_fixture():
    engine = create_engine("sqlite://",                    # in-memory
        connect_args={"check_same_thread": False}, poolclass=StaticPool)
    SQLModel.metadata.create_all(engine)                   # empty team + hero
    with Session(engine) as session:
        yield session

@pytest.fixture(name="client")
def client_fixture(session):
    def get_session_override():
        return session
    app.dependency_overrides[get_session] = get_session_override
    yield TestClient(app)
    app.dependency_overrides.clear()
```

- Every test gets an **empty, separate** DB; use session to insert heroes, client to call the API.
- **Limitation**: SQLite does not enforce foreign keys by default (PostgreSQL does) – check such rules in code.

Test the Error Paths Too

```python
def test_read_hero_not_found(client):
    r = client.get("/api/v1/heroes/999")
    assert r.status_code == 404
    assert r.json() == {"detail": "Hero not found"}

@pytest.mark.parametrize("payload", [
    {}, {"name": "X"},                                  # missing fields
    {"name": "X", "secret_name": "y", "age": "old"},    # wrong type
    {"name": "X", "secret_name": "y", "age": -5},       # ge=0
    {"name": "X", "secret_name": "y", "status": "dead"}, # not an enum value
])
def test_create_hero_invalid(client, payload):
    assert client.post("/api/v1/heroes", json=payload).status_code == 422
```

```
python -m pytest -v      # -k name filter, -x stop at first failure
```

Frontend Calls the Heroes API

fetch(): **List Heroes**

```
const API = "http://127.0.0.1:8000/api/v1";
const tbody = document.querySelector("#heroes tbody");

async function loadHeroes() {
  const res = await fetch(`${API}/heroes?limit=50`);
  const heroes = await res.json();          // array of HeroPublic objects
  tbody.replaceChildren();                  // clear old rows first
  for (const h of heroes) {
    const tr = tbody.insertRow();
    for (const v of [h.id, h.name, h.age ?? "-", h.status])
      tr.insertCell().textContent = v;      // textContent, NOT innerHTML
  }
}
```

- Try GET /api/v1/heroes in /docs **first**, then wire it into the page.
- A hero named <img onerror=...> via innerHTML ⇒ **XSS** (Week 11).

fetch(): **Create a Hero & Show 422 Errors**

```
async function createHero(data) {
  const res = await fetch(`${API}/heroes`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),              // JS object -> JSON text
  });
  if (res.status === 201) {
    await loadHeroes();                      // refresh the table
  } else if (res.status === 422) {
    const err = await res.json();
    alert(err.detail.map(d => `${d.loc.at(-1)}: ${d.msg}`).join("\n"));
  } else {
    alert(`Error ${res.status}`);
  }
}
```

- createHero({name: "X", secret_name: "y", age: -5}) ⇒ "age: Input should be greater than or equal to 0".
- loc from the 422 body tells the page **which field** is wrong.

JSON Is the Contract

The page reads h.id, h.name, h.age, h.status. Now the backend renames a field:

```python
class HeroBase(SQLModel):
    years: int | None = None     # was: age
```

- API works, /docs looks fine – but every Age cell shows "-" (h.age is undefined). **Nothing crashes, nobody notices.**
- Field names/types are an **agreement**: announce every schema change.
- Check /docs before writing fetch() code.
- Breaking change to a public API ⇒ new version (/api/v2).

API Quality

API Quality: The Heroes API Checklist

- **Consistent names**: snake_case (team_id, created_at); every hero response = HeroPublic.
- **Consistent errors**: {"detail": "Hero not found"}; standard 422 format.
- **Pagination & filter**: /heroes?team_id=1&offset=0&limit=10, bounded with Query(ge=1, le=100).
- **Versioning**: include_router(heroes.router, prefix="/api/v1"), same for teams. Itself a breaking change (/heroes → /api/v1/heroes) – add it before a frontend depends on the API.
- **Status codes**: 201 create, 204 delete, 400 unknown team_id, 404 missing hero/team, 409 duplicate team, 422 invalid body.
- **Documented** (tags, summary, examples) and **tested** (success + error per endpoint).

Troubleshooting by status

404 forgot /api/v1? /hero vs /heroes? **422** read detail[].loc **500** read the uvicorn traceback **CORS** origin not in allow_origins

Strategy for DS & AI Students

Learning Strategy

- /docs **is the "wow"**: add status, refresh – schema, drop-down and test form update instantly.
- pytest **is an old friend**: you test data functions already; TestClient adds HTTP.
- **Classify URLs**: rewrite an RPC-style Heroes API as REST (Lab 1).
- **Close the loop**: JSON = contract, /docs = its documentation, tests = its guarantee.

Worked Example & Labs

Worked Example: From Week 9 to Week 10

```
project/
|-- app/
|   |-- database.py          # (Week 9, unchanged)
|   |-- models.py            # + rules, examples, status, created_at, Team schemas
|   |-- routers/heroes.py    # NEW: Week 9 hero endpoints + docs + team_id filter
|   |-- routers/teams.py     # NEW: GET/POST /teams (409), /teams/{id}/heroes
|   |-- main.py              # metadata, CORS, include_router("/api/v1")
|   `-- seed.py              # (Week 9, unchanged)
|-- tests/conftest.py        # NEW: test DB + client fixtures
|-- tests/test_heroes.py     # NEW
|-- tests/test_teams.py      # NEW
|-- frontend/index.html      # NEW: fetch() client
`-- pytest.ini               # pythonpath = .   testpaths = tests
```

Worked Example: models.py – Team

```python
class TeamBase(SQLModel):
    name: str = Field(index=True, unique=True, min_length=1, max_length=50,
                      schema_extra={"examples": ["Avengers"]})
    headquarters: str = Field(schema_extra={"examples": ["New York"]})

class Team(TeamBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    heroes: list["Hero"] = Relationship(back_populates="team")

class TeamCreate(TeamBase):
    pass

class TeamPublic(TeamBase):
    id: int
```

- Team now follows the same Base / Create / Public pattern as Hero (Week 9).

Worked Example: models.py – Hero

```python
class HeroBase(SQLModel):
    name: str = Field(index=True, min_length=1, max_length=50,
                      schema_extra={"examples": ["Deadpond"]})
    age: int | None = Field(default=None, ge=0, le=1000,
                            schema_extra={"examples": [30]})
    status: HeroStatus = HeroStatus.active
    team_id: int | None = Field(default=None, foreign_key="team.id")

class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    secret_name: str
    created_at: datetime = Field(default_factory=utc_now)
    team: Team | None = Relationship(back_populates="heroes")

class HeroCreate(HeroBase):
    secret_name: str = Field(schema_extra={"examples": ["Dive Wilson"]})

class HeroPublic(HeroBase):
    id: int
    created_at: datetime
```

HeroUpdate: **Optional ≠ Nullable**

```python
class HeroUpdate(SQLModel):
    name: str | None = Field(default=None, min_length=1, max_length=50)
    age: int | None = Field(default=None, ge=0, le=1000)
    status: HeroStatus | None = None
    team_id: int | None = None              # null = remove the hero from its team

    @field_validator("name", "status")
    @classmethod
    def not_null(cls, value):
        # Only runs when the client sends the field
        if value is None:
            raise ValueError("cannot be null")
        return value
```

- Optional lets the client send only what changes – but also send {"name": null}.
- name/status are NOT NULL columns ⇒ without the validator: **500**; with it: **422**.

Worked Example: routers/heroes.py **(1/2)**

```python
router = APIRouter(prefix="/heroes", tags=["heroes"])
NOT_FOUND = {404: {"description": "Hero not found"}}

def get_hero_or_404(session, hero_id: int) -> Hero:
    hero = session.get(Hero, hero_id)
    if not hero:
        raise HTTPException(status_code=404, detail="Hero not found")
    return hero

def check_team_exists(session, team_id: int | None) -> None:
    if team_id is not None and not session.get(Team, team_id):
        raise HTTPException(status_code=400, detail=f"Team {team_id} does not exist")

@router.get("", response_model=list[HeroPublic], summary="List heroes")
def list_heroes(session: SessionDep, team_id: int | None = None,
                offset: int = Query(default=0, ge=0),
                limit: int = Query(default=10, ge=1, le=100)):
    query = select(Hero).order_by(Hero.id)
    if team_id is not None:
        query = query.where(Hero.team_id == team_id)
    return session.exec(query.offset(offset).limit(limit)).all()
```

- Without ge=1, ?limit=-1 means “no limit” in SQLite – the cap is bypassed.

Worked Example: routers/heroes.py **(2/2)**

```python
@router.post("", response_model=HeroPublic, status_code=status.HTTP_201_CREATED,
             summary="Create a hero",
             responses={400: {"description": "Team does not exist"}})
def create_hero(hero_in: HeroCreate, session: SessionDep):
    check_team_exists(session, hero_in.team_id)
    hero = Hero.model_validate(hero_in)
    session.add(hero); session.commit(); session.refresh(hero)
    return hero

@router.patch("/{hero_id}", response_model=HeroPublic, summary="Update a hero partially",
              responses={**NOT_FOUND, 400: {"description": "Team does not exist"}})
def update_hero(hero_id: int, hero_in: HeroUpdate, session: SessionDep):
    hero = get_hero_or_404(session, hero_id)
    changes = hero_in.model_dump(exclude_unset=True)   # only fields the client sent
    check_team_exists(session, changes.get("team_id"))
    hero.sqlmodel_update(changes)
    session.add(hero); session.commit(); session.refresh(hero)
    return hero
# + GET /{hero_id}, DELETE /{hero_id} (204) --- Week 9 code, now using get_hero_or_404
```

- Unknown team_id: PostgreSQL rejects the FK ⇒ **500**; SQLite silently stores it. check_team_exists ⇒ **400**.

Worked Example: routers/teams.py

```python
router = APIRouter(prefix="/teams", tags=["teams"])

@router.post("", response_model=TeamPublic, status_code=status.HTTP_201_CREATED,
             summary="Create a team",
             responses={409: {"description": "Team name already exists"}})
def create_team(team_in: TeamCreate, session: SessionDep):
    team = Team.model_validate(team_in)
    session.add(team)
    try:
        session.commit()
    except IntegrityError:                      # violates unique=True on name
        session.rollback()
        raise HTTPException(status_code=409, detail="Team name already exists")
    session.refresh(team)
    return team

@router.get("/{team_id}/heroes", response_model=list[HeroPublic],
            summary="List the heroes of a team",
            responses={404: {"description": "Team not found"}})
def list_team_heroes(team_id: int, session: SessionDep):
    team = session.get(Team, team_id)
    if not team:
        raise HTTPException(status_code=404, detail="Team not found")
    return team.heroes                          # the Week 9 Relationship
# + GET "" -> list teams
```

- Without the `try`, a duplicate name would be a **500**; **409** says what went wrong.

Worked Example: main.py & the Final API

```python
app.add_middleware(CORSMiddleware,
    allow_origins=["http://localhost:5500", "http://127.0.0.1:5500"],
    allow_methods=["*"], allow_headers=["*"])
app.include_router(heroes.router, prefix="/api/v1")
app.include_router(teams.router, prefix="/api/v1")
```

Method	URL (under /api/v1)	OK	Errors
GET	/heroes?team_id=&offset=&limit=	200	422
GET / DELETE	/heroes/{hero_id}	200 / 204	404
PATCH	/heroes/{hero_id}	200	400, 404, 422
POST	/heroes	201	400, 422
GET / POST	/teams	200 / 201	409, 422
GET	/teams/{team_id}/heroes	200	404

Worked Example: Tests

```python
def test_create_hero(client):
    r = client.post("/api/v1/heroes",
                    json={"name": "Deadpond", "secret_name": "Dive Wilson"})
    assert r.status_code == 201
    assert r.json()["status"] == "active"        # Enum -> value
    assert "secret_name" not in r.json()         # hidden by response_model

def test_update_hero_partial(session, client):
    hero = Hero(name="Tony", secret_name="Iron Man", age=45)
    session.add(hero); session.commit()
    r = client.patch(f"/api/v1/heroes/{hero.id}", json={"status": "retired"})
    assert r.json()["age"] == 45 and r.json()["status"] == "retired"

def test_create_team_duplicate(client):
    body = {"name": "Avengers", "headquarters": "New York"}
    assert client.post("/api/v1/teams", json=body).status_code == 201
    assert client.post("/api/v1/teams", json=body).status_code == 409

def test_hero_unknown_team(client):
    r = client.post("/api/v1/heroes",
                    json={"name": "A", "secret_name": "a", "team_id": 999})
    assert r.status_code == 400
```

```
python -m pytest -v                  # 23 tests pass, real DB untouched
rm -f app.db && python -m app.seed   # new columns (dev only)
fastapi dev app/main.py              # /docs, /redoc, then the Live Server page
```

Lab 1: Completing REST

Hands-on practice (Heroes API)

- Rewrite as REST: GET /getHeroes, GET /heroById?id=3, POST /addTeam, POST /removeHero/3, GET /heroesOfTeam?team=1, POST /changeHeroTeam?hero=3&team=2.
- Move hero endpoints into routers/heroes.py: tags, summary, docstrings, responses={404: ...}; mount under /api/v1.
- Add validation rules + examples to HeroBase/HeroUpdate; add status and created_at; recreate the DB.
- Create routers/teams.py: GET/POST /teams (409), GET /teams/{id}/heroes.
- Refresh /docs and /redoc: two sections, titles, examples, status drop-down.

Lab 2: CORS + Fetch

Hands-on practice (Heroes API)

- Add `CORSMiddleware` with your Live Server origin(s).
- `frontend/index.html`: `GET /api/v1/heroes`, render a table with `textContent`.
- Form that `POST`s a hero; refresh on `201`, show field errors on `422` (try age `-5`).
- Remove your origin once to see the CORS error – then restore it.
- Extension: team `<select>` from `GET /teams`; show `GET /teams/{id}/heroes`.

Lab 3: Testing + Project Integration

Hands-on practice

- pytest.ini + conftest.py: in-memory SQLite session + client with dependency_overrides.
- ≥**4 hero tests**: GET list (200), GET detail (200 & 404), POST (201), POST invalid (422); then unknown team_id (400), teams: 409 and nested 200/404.
- python -m pytest -v until green; confirm the real app.db is unchanged.
- Apply the same steps to your **group project's** API; share its /docs as the frontend contract.

Review & Summary

Quick MCQ Review

1. REST endpoint for hero id 5 details? GET /heroes/5 (not GET /getHero?id=5)
2. DELETE /heroes/5 succeeded, no content – which code? **204 No Content**
3. Where does /docs come from? **The OpenAPI spec FastAPI generates from code**
4. Which side configures CORS? **The server/backend – it declares allowed origins**
5. What does TestClient let you do? **Call endpoints in tests without a running server**
6. Which component validates & serializes JSON? **Pydantic (SQLModel) models**
7. POST /heroes with "age": -5, ge=0? **422,** loc: ["body", "age"]
8. CORS config to never use? allow_origins=["*"] + allow_credentials=True
9. Keep tests off the real DB? **Test DB +** app.dependency_overrides[get_session]
10. Page 127.0.0.1:5500 calls API 127.0.0.1:8000 – cross-origin? **Yes – port differs**
11. POST /teams with an existing (unique) name? **409 Conflict (catch** IntegrityError**)**
12. Why /api/v1? **Versioning – breaking changes go to** /api/v2

Self-Check Checklist

- ☐ Can design REST URLs for heroes/teams and spot RPC-style ones
- ☐ Know when to return 200, 201, 204, 400, 401, 403, 404, 409, 422, 500
- ☐ Can add validation rules and control output with `response_model`
- ☐ Can improve `/docs` with tags, summary, docstrings, examples
- ☐ Can configure `CORSMiddleware` and explain what an origin is
- ☐ Can write `pytest` + `TestClient` tests on a separate test DB
- ☐ Can list and create heroes from a web page with `fetch()`

Summary and Next Session

- **REST**: plural nouns (/heroes, /teams), HTTP verbs, stateless, correct status codes; verbs in URLs = **RPC**; “retire a hero” = PATCH.
- **Pydantic**: Field rules ⇒ automatic **422**; response_model=HeroPublic controls output; datetime → ISO 8601, Enum → value.
- **OpenAPI** (/openapi.json) → **Swagger UI** (/docs) and **ReDoc** (/redoc); routers with tags, summary, examples.
- **CORS**: browser rule, fixed on the **server** with CORSMiddleware + explicit origins.
- pytest + TestClient: fixtures, in-memory test DB, dependency_overrides; test 400/404/409/422 too.
- Hero page with fetch() closes the loop – **JSON is the contract**.
- Quality: consistent names/errors, pagination & filter, /api/v1 versioning.
- Next (Week 11): authentication & authorization – only logged-in users may create/delete heroes; 401/403.

Q&A

Questions & Answers